

Context-Free Languages & Grammars (CFLs & CFGs)

1. Introduction to Context-Free Languages

1.1 Why Context-Free Languages?

Not all languages can be recognized by regular expressions or finite automata. There are languages that require more powerful mechanisms to describe and recognize them.

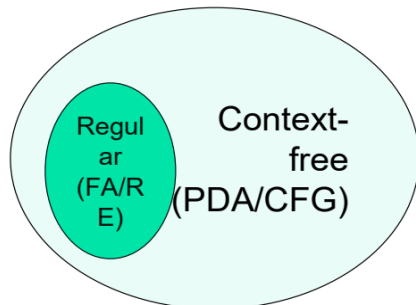
Key Question: If a language is not regular, can we still create a recognizer for it?

Answer: Yes! Context-Free Grammars (CFGs) provide this capability.

1.2 Relationship Between Language Classes

Context-Free Languages form a larger class than Regular Languages:

- Every Regular Language is a Context-Free Language



- But NOT every Context-Free Language is Regular
- Think of it as nested circles: Regular languages are inside the larger circle of Context-Free languages

2. Motivation: The Palindrome Example

2.1 What is a Palindrome?

A palindrome is a word that reads the same forwards and backwards.

Examples:

- Text: "madam", "redivider", "malayalam"
- Binary: "010010010", "1001001", "11011"

2.2 Why Palindromes Are Not Regular

Let's consider the language $L = \{w \mid w \text{ is a binary palindrome}\}$

Any long enough string can be cut into 3 parts and a middle part can be repeated or removed, and the string will still stay in the language

Proof that L is NOT regular (using Pumping Lemma):

1. Assume L is regular with pumping length N
2. Consider the string $w = 0^N 1 0^N$ (N zeros, one 1, N zeros)
3. This is a palindrome, so w must be in L
4. By the Pumping Lemma, we can write $w = xyz$ where:
 - $|xy| \leq N$
 - $|y| > 0$
 - $xy^k z$ must be in L for all $k \geq 0$
5. Since $|xy| \leq N$, both x and y consist only of 0's from the first part
6. Therefore $y = 0^m$ for some $m > 0$
7. If we pump with $k = 0$ (removing y), we get xz
8. This gives us $0^{(N-m)} 1 0^N$, which is NOT a palindrome
9. **Contradiction!** Therefore, L is not regular.

2.3 Palindromes as a Context-Free Language

However, palindromes CAN be described using a CFG:

Grammar for Binary Palindromes:

Rule 1: $A \rightarrow \epsilon$ (empty string is a palindrome)

Rule 2: $A \rightarrow 0$ (single 0 is a palindrome)

Rule 3: $A \rightarrow 1$ (single 1 is a palindrome)

Rule 4: $A \rightarrow 0A0$ (wrap a palindrome with 0s)

Rule 5: $A \rightarrow 1A1$ (wrap a palindrome with 1s)

Compact notation: $A \rightarrow 0A0 \mid 1A1 \mid 0 \mid 1 \mid \epsilon$

Example: Generating "01110"

Step 1: Start with A

Step 2: Apply Rule 4: A becomes 0A0

Step 3: Apply Rule 5: 0A0 becomes 01A10

Step 4: Apply Rule 3: 01A10 becomes 01110

Written as: $A \Rightarrow 0A0 \Rightarrow 01A10 \Rightarrow 01110$

3. Formal Definition of Context-Free Grammar

3.1 Components of a CFG

Context-free grammars (CFGs) are used to describe **context-free languages**. A context-free grammar is a set of recursive rules used to generate patterns of strings. A context-free grammar can describe all regular languages and more, but they cannot describe *all* possible languages.

A Context-Free Grammar G is defined as a 4-tuple: $G = (V, T, P, S)$

Where:

V = Set of Variables (also called Non-terminals)

- These are symbols that can be replaced by other strings
- Convention: Use uppercase letters A, B, C, S, etc.

T = Set of Terminals (the actual alphabet symbols)

- These are the final symbols that appear in strings
- Convention: Use lowercase letters a, b, c, or digits 0, 1, 2, etc.
- Can include ϵ (empty string)

P = Set of Productions (rules for replacement)

- Format: $V \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$
- Each α_i is a string of variables and terminals

S = Start Variable (where all derivations begin)

- Must be an element of V

3.2 Example: Formal CFG for Binary Palindromes

$G = (\{A\}, \{0, 1\}, P, A)$

Where:

- $V = \{A\}$ (only one variable)
- $T = \{0, 1\}$ (two terminals)
- $S = A$ (start variable)
- $P: A \rightarrow 0A0 \mid 1A1 \mid 0 \mid 1 \mid \epsilon$

4. Common CFG Examples

4.1 Example 1: Equal Numbers of 0s and 1s

Language: $L = \{0^n 1^n \mid n \geq 1\}$

This language contains strings like: 01, 0011, 000111, 00001111, etc.

CFG:

$S \rightarrow 01$

$S \rightarrow 0S1$

How it works:

- Basis: "01" is in the language
- Induction: If w is in the language, then $0w1$ is also in the language

Derivation of "000111":

Step 1: S

Step 2: $S \Rightarrow 0S1$ (apply second rule)

Step 3: $0S1 \Rightarrow 00S11$ (apply second rule again)

Step 4: $00S11 \Rightarrow 000111$ (apply first rule)

Complete: $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000111$

4.2 Example 2: Balanced Parentheses

Language: Strings with properly balanced parentheses like $()$, $((()))$, $((()()))$, etc.

CFG:

$S \rightarrow (S)$ (wrap balanced parens around another balanced string)

$S \rightarrow SS$ (concatenate two balanced strings)

$S \rightarrow \epsilon$ (empty string is balanced)

Derivation of "((()))()":

Step 1: S

Step 2: $S \Rightarrow SS$ (two parts)

Step 3: $SS \Rightarrow (S)S$ (wrap first part)

Step 4: $(S)S \Rightarrow ((S))S$ (wrap again)

Step 5: $((S))S \Rightarrow (((S)))S$ (wrap once more)

Step 6: $(((((S))))S \Rightarrow (((()))S$ (replace S with ϵ)

Step 7: $((()))S \Rightarrow ((()))(S)$ (wrap second part)

Step 8: $((()))(S) \Rightarrow ((()))(\epsilon)$ (replace S with ϵ)

Step 9: $((()))(\epsilon) \Rightarrow ((()))()$ (simplify)

4.3 Example 3: More 0s than 1s

Language: $L = \{0^m 1^n \mid m \geq n\}$

This allows strings like: 0, 00, 01, 001, 0001, 00011, etc.

CFG:

$S \rightarrow 0S1$ (equal 0s and 1s, but we can add more 0s)

$S \rightarrow A$ (switch to adding extra 0s)

$A \rightarrow 0A$ (add more 0s) $A \rightarrow \epsilon$ (done)

Derivation of "00000111":

Step 1: S

Step 2: $S \Rightarrow 0S1$

Step 3: $0S1 \Rightarrow 00S11$

Step 4: $00S11 \Rightarrow 000S111$

Step 5: $000S111 \Rightarrow 000A111$ (switch to A)

Step 6: $000A111 \Rightarrow 0000A111$ (add extra 0)

Step 7: $0000A111 \Rightarrow 00000111$ (finish with ϵ)

4.4 Example 4: If-Then-Else Statements

Language: Programming constructs with optional else clause

CFG:

$S \rightarrow \text{if Condition then Statement else Statement}$

$S \rightarrow \text{if Condition then Statement}$

This can be extended with proper definitions for Condition and Statement.

4.5 Additional Examples

Example A: $L_1 = \{0^n \mid n \geq 0\}$ (any number of 0s including empty)

$A \rightarrow 0A \mid \epsilon$

Example B: $L_2 = \{0^n \mid n \geq 1\}$ (at least one 0)

$A \rightarrow 0A \mid 0$

Example C: $L_3 = \{0^i 1^j 2^k \mid i = j \text{ OR } j = k, i, j, k \geq 0\}$

$S \rightarrow AB \mid C$

$A \rightarrow 0A1 \mid \epsilon$ (generates equal 0s and 1s)

$B \rightarrow 2B \mid \epsilon$ (generates any number of 2s)

$C \rightarrow 0C \mid D$ (generates any number of 0s)

$D \rightarrow 1D2 \mid \epsilon$ (generates equal 1s and 2s)

5. Derivations in CFGs

5.1 What is a Derivation?

A derivation is a sequence of steps that shows how to generate a string from the start symbol using the production rules.

Notation:

- $\alpha \Rightarrow \beta$ means " α derives β in one step"
- $\alpha \Rightarrow^* \beta$ means " α derives β in zero or more steps"

5.2 Formal Definition

If we have a production $A \rightarrow \gamma$, then: $\alpha A \beta \Rightarrow \alpha \gamma \beta$ (we can replace A with γ anywhere it appears)

Example with $S \rightarrow 01$ and $S \rightarrow 0S1$:

$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000111$

5.3 Transitivity of Derivations

If $A \Rightarrow^* B$ and $B \Rightarrow^* C$, then $A \Rightarrow^* C$

This means derivations can be chained together.

6. Context-Free Language Definition

6.1 Language of a Grammar

Context-Free Language (CFL) is a set of strings defined by a **Context-Free Grammar (CFG)**, allowing for the description of nested structures like programming language syntax that are too complex for simpler regular languages. The language of a CFG $G = (V, T, P, S)$, denoted $L(G)$, is:

$$L(G) = \{w \in T \mid S \Rightarrow^* w\}$$

In plain English: The language is the set of all terminal strings that can be derived from the start symbol.

Important points:

- Only terminal strings (no variables) are in the language
- Must be derivable from the start symbol
- All strings in T^* that can be generated form the language

7. Leftmost and Rightmost Derivations

7.1 Definitions

Leftmost Derivation: At each step, replace the leftmost variable in the current string.

Rightmost Derivation: At each step, replace the rightmost variable in the current string.

7.2 Example: Expression Grammar

Grammar:

$E \rightarrow E + E \mid E * E \mid (E) \mid F$

$F \rightarrow aF \mid bF \mid 0F \mid 1F \mid a \mid b \mid 0 \mid 1$

*Deriving "a(ab+10)":**

LEFTMOST DERIVATION (always replace leftmost variable first):

$E \Rightarrow E * E$ (replace leftmost E)

$\Rightarrow F * E$ (replace leftmost E)

$\Rightarrow aF * E$ (replace F)

$\Rightarrow a * E$ (replace F with ϵ)

$\Rightarrow a * (E)$ (replace rightmost E)

$\Rightarrow a * (E + E)$ (continue...)

$\Rightarrow a * (F + E)$

$\Rightarrow a * (aF + E)$

$\Rightarrow a * (abF + E)$

$\Rightarrow a * (ab + E)$

$\Rightarrow a * (ab + F)$

$\Rightarrow a * (ab + 1F)$

$\Rightarrow a * (ab + 10F)$

$\Rightarrow a * (ab + 10)$

RIGHTMOST DERIVATION (always replace rightmost variable first):

$E \Rightarrow E * E$ (start)

$\Rightarrow E * (E)$ (replace rightmost E)

$\Rightarrow E * (E + E)$ (replace E in parens)

$\Rightarrow E * (E + F)$ (replace rightmost E)

$\Rightarrow E * (E + 1F)$ (continue...)

$\Rightarrow E * (E + 10F)$

$\Rightarrow E * (E + 10)$

$\Rightarrow E * (F + 10)$

$\Rightarrow E * (aF + 10)$

$\Rightarrow E * (abF + 10)$

$\Rightarrow E * (ab + 10)$

$\Rightarrow F * (ab + 10)$

$\Rightarrow aF * (ab + 10)$

$\Rightarrow a * (ab + 10)$

7.3 Important Properties

Q1: For every leftmost derivation, is there a rightmost derivation?

Answer: TRUE. We can prove this using parse trees.

Q2: Does every word generated by a CFG have both leftmost and rightmost derivations?

Answer: YES. Easy to prove by reversing the derivation.

Q3: Can a word have more than one leftmost (or rightmost) derivation?

Answer: YES, depending on the grammar. This is called ambiguity.

8. Parse Trees

8.1 Definition

A parse tree is a tree representation of a derivation with these properties:

1. **Leaves:** Labeled by terminals or ϵ
2. **Interior nodes:** Labeled by variables

3. **Root:** Labeled by the start symbol
4. **Children:** For a production $A \rightarrow X_1X_2...X_k$, node A has k children labeled $X_1, X_2, ..., X_k$ (left to right)

8.2 Yield of a Parse Tree

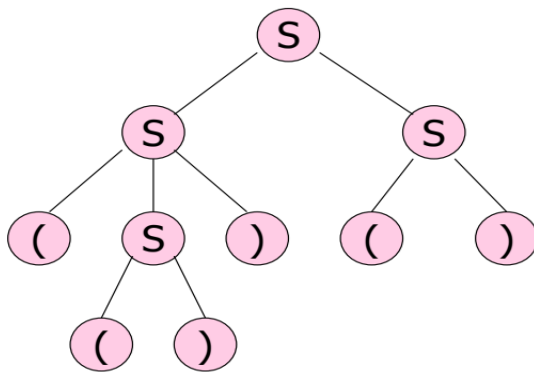
The **yield** is the concatenation of leaf labels read from left to right (preorder traversal).

8.3 Example 1: Balanced Parentheses

Grammar: $S \rightarrow SS \mid (S) \mid ()$

Parse tree for "(()())":

Tree Structure:



Reading leaves left to right: ((()) ()

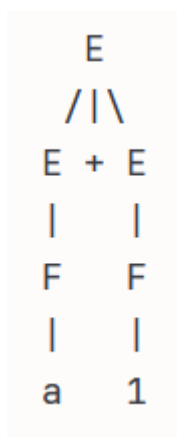
Yield: (()())

8.4 Example 2: Simple Expression

Grammar: $E \rightarrow E + E \mid F, F \rightarrow a \mid 1$

Parse tree for "a + 1":

Tree Structure:



Reading leaves left to right: $a + 1$

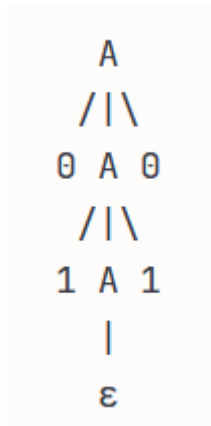
Yield: $a + 1$

8.5 Example 3: Binary Palindrome

Grammar: $A \rightarrow 0A0 \mid 1A1 \mid 0 \mid 1 \mid \epsilon$

Parse tree for "0110":

Tree Structure:



Reading leaves left to right: $0\ 1\ \epsilon\ 1\ 0 = 0110$

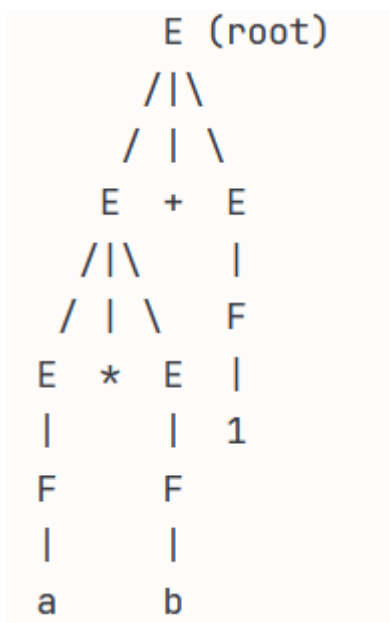
Yield: 0110

8.6 Example 4: Expression with Multiple Operators

Grammar: $E \rightarrow E + E \mid E * E \mid F, F \rightarrow a \mid b \mid 1$

Parse tree for " $a * b + 1$ ":

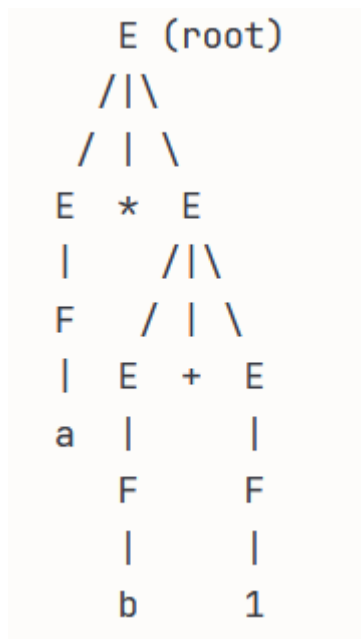
Tree Structure (Ambiguous - Version 1):



This gives: $(a * b) + 1$

Reading leaves: $a * b + 1$

Tree Structure (Ambiguous - Version 2):



This gives: $a * (b + 1)$

Reading leaves: $a * b + 1$

Note: Same string " $a * b + 1$ " but two different tree structures showing ambiguity!

9. Ambiguity in Context-Free Grammars

9.1 Definition of Ambiguity

A CFG is **ambiguous** if there exists at least one string that has:

- More than one leftmost derivation, OR
- More than one rightmost derivation, OR
- More than one parse tree

All three conditions are equivalent.

9.2 Example 1: Simple Ambiguous Grammar

Grammar: $S \rightarrow AS \mid \epsilon$ $A \rightarrow A1 \mid 0A1 \mid 01$

String: "00111"

Leftmost Derivation #1:

Step 1: S

Step 2: $S \Rightarrow AS$

Step 3: $AS \Rightarrow 0A1S$ (using $A \rightarrow 0A1$)

Step 4: $0A1S \Rightarrow 0A11S$ (using $A \rightarrow A1$)

Step 5: $0A11S \Rightarrow 00111S$ (using $A \rightarrow 01$)

Step 6: $00111S \Rightarrow 00111$ (using $S \rightarrow \epsilon$)

Leftmost Derivation #2:

Step 1: S

Step 2: $S \Rightarrow AS$

Step 3: $AS \Rightarrow A1S$ (using $A \rightarrow A1$)

Step 4: $A1S \Rightarrow 0A11S$ (using $A \rightarrow 0A1$)

Step 5: $0A11S \Rightarrow 00111S$ (using $A \rightarrow 01$)

Step 6: $00111S \Rightarrow 00111$ (using $S \rightarrow \epsilon$)

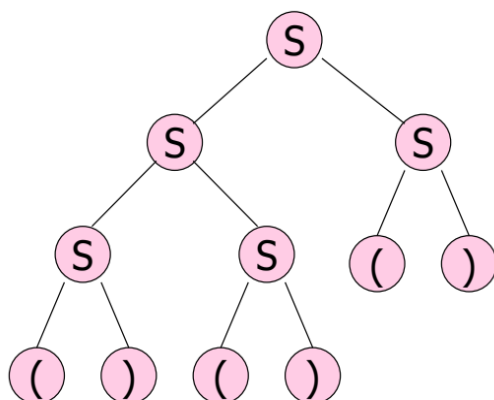
Conclusion: Two different leftmost derivations exist, so the grammar is ambiguous!

9.3 Example 2: Balanced Parentheses

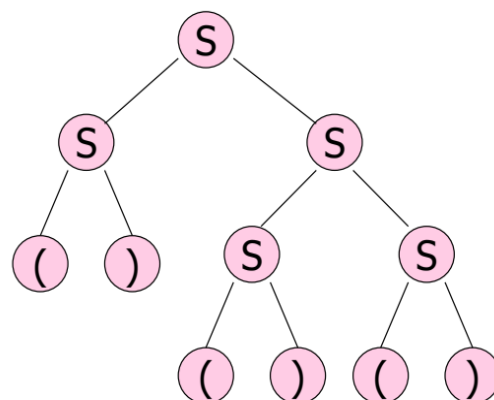
Grammar: $S \rightarrow SS \mid (S) \mid ()$

String: "()()()"

Parse Tree #1:



Parse Tree #2



Conclusion: Different tree structures for same string, so this grammar is ambiguous!

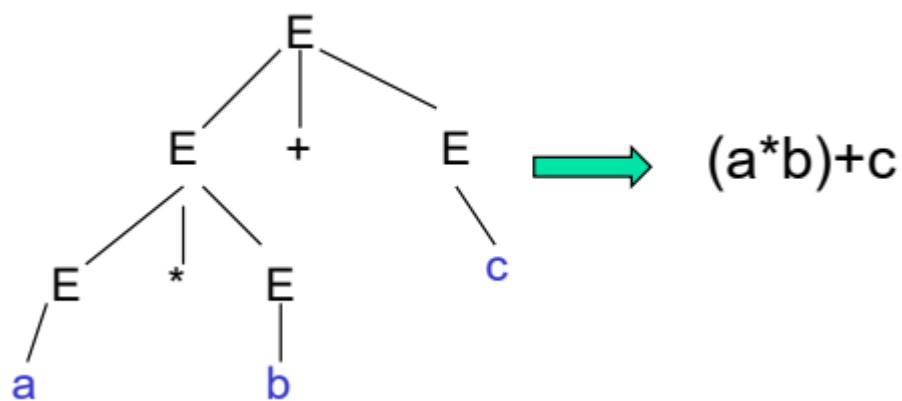
9.4 Why Ambiguity Matters: Expression Evaluation

Grammar: $E \rightarrow E + E \mid E * E \mid (E) \mid a \mid b \mid c \mid 0 \mid 1$

String: "a * b + c"

Parse Tree #1 gives (a*b)+c:

Tree Structure 1:



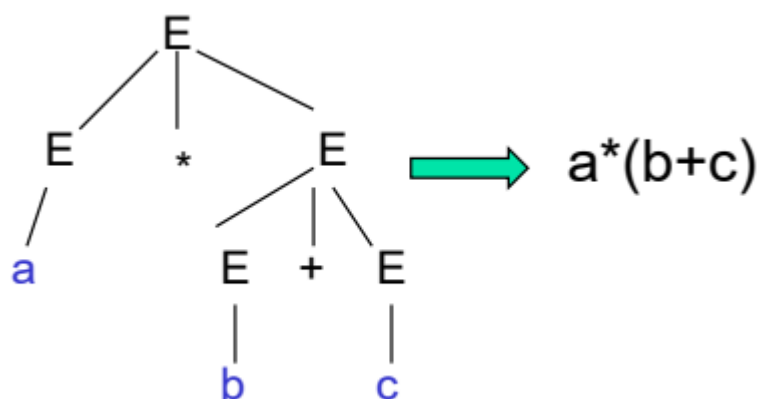
Evaluation: First compute $a*b$, then add c

Result: $(a*b) + c$

If $a=2, b=3, c=4$: $(2*3)+4 = 6+4 = 10$

Parse Tree #2 gives $a(b+c)$:

Tree Structure 2:



Evaluation: First compute $b+c$, then multiply by a

Result: $a * (b+c)$

If $a=2, b=3, c=4$: $2*(3+4) = 2*7 = 14$

Problem: Different parse trees give different calculated values!

- Tree 1 gives: 10
- Tree 2 gives: 14

This is a serious issue in compilers and interpreters. We need unambiguous grammars!

10. Removing Ambiguity

10.1 Can Ambiguity Be Removed?

For **some** CFLs, we can rewrite the grammar to remove ambiguity. For **other** CFLs, it's impossible (inherently ambiguous).

10.2 Expression Grammar Example

AMBIGUOUS VERSION:

$$E \rightarrow E + E \mid E * E \mid (E) \mid a \mid b \mid c \mid 0 \mid 1$$

UNAMBIGUOUS VERSION (with precedence):

$$E \rightarrow E + T \mid T \text{ (addition has lowest precedence)}$$
$$T \rightarrow T * F \mid F \text{ (multiplication has higher precedence)}$$
$$F \rightarrow I \mid (E) \text{ (parentheses and identifiers highest)}$$
$$I \rightarrow a \mid b \mid c \mid 0 \mid 1 \text{ (basic identifiers)}$$

Precedence Order: Parentheses $()$ > Multiplication $*$ > Addition $+$

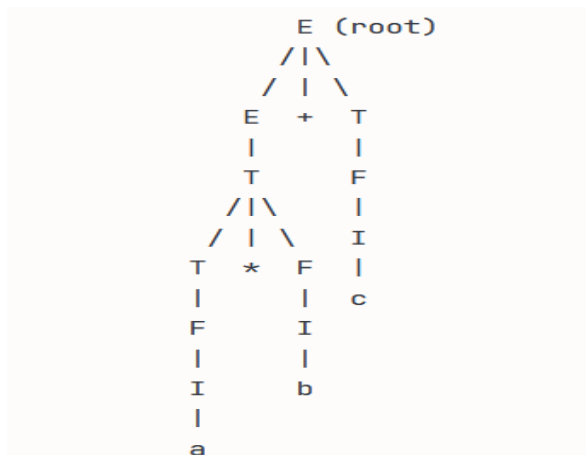
10.3 How This Removes Ambiguity

The new grammar enforces:

1. Multiplication binds tighter than addition
2. Parentheses override this precedence
3. Only ONE parse tree possible for any expression

Example: "a*b+c" with unambiguous grammar has ONLY ONE parse tree:

Tree Structure (ONLY possibility):



Evaluation Path:

1. Bottom-up: a is I is F is T
2. b is I is F
3. T * F gives T (a*b)
4. c is I is F is T
5. E + T gives E ((a*b)+c)

Result: (a*b)+c (correct mathematical precedence)

If a=2, b=3, c=4: (2*3)+4 = 10

Key Insight: The grammar structure forces multiplication before addition, giving only one interpretation!

11. Inherently Ambiguous Languages

11.1 Definition

A CFL L is **inherently ambiguous** if **every** CFG that generates L is ambiguous.

No matter how you write the grammar, it will always be ambiguous for at least some strings.

11.2 Example: Union of Two Patterns

Language:

$$L = \{a^n b^n c^m d^m \mid n, m \geq 1\} \cup \{a^n b^m c^m d^n \mid n, m \geq 1\}$$

In plain English:

PART 1: Equal a's and b's, followed by equal c's and d's Examples: abcd, aabbccdd, aaabbbccddddd

PART 2: Equal a's and d's, with equal b's and c's in the middle Examples: abcd, aabccdd, aaabccdd

Why it's inherently ambiguous:

Consider the special string: $a^n b^n c^n d^n$ (all four symbols appear n times)

This string matches BOTH patterns:

1. It has n a's and n b's (first part), then n c's and n d's (second part) ✓
2. It has n a's and n d's (outer), with n b's and n c's (inner) ✓

One Possible Grammar (using 0,1,2 instead of a,b,c,d for simplicity):

$S \rightarrow AB \mid CD$ $A \rightarrow 0A1 \mid 01$ (equal 0s and 1s) $B \rightarrow 2B \mid 2$ (any positive number of 2s) $C \rightarrow 0C \mid 0$ (any positive number of 0s) $D \rightarrow 1D2 \mid 12$ (equal 1s and 2s)

Example of ambiguity for "012":

Derivation Path 1: $S \Rightarrow AB \Rightarrow 01B \Rightarrow 012$

Derivation Path 2: $S \Rightarrow CD \Rightarrow 0D \Rightarrow 012$

Conclusion: Every grammar for this language will have similar ambiguity for strings where all counts are equal. This is INHERENTLY AMBIGUOUS.

14.2 Key Differences: Regular vs Context-Free Languages

COMPARISON TABLE:

ASPECT	REGULAR LANGUAGES	CONTEXT-FREE LANGUAGES
Power	Less powerful	More powerful
Notation	Regular Expressions	Context-Free Grammars
Recognizer	Finite Automata (FA)	Pushdown Automata (PDA)
Recursion	No recursion allowed	Supports recursion
Counting	Cannot count	Can count/match pairs
Example	$(a+b)^*$	$\{a^n b^n\}$
Nesting	Cannot handle nesting	Handles nested structures
Memory	No memory (states only)	Stack memory available

15. Step-by-Step Problem-Solving Guide

15.1 Writing a CFG for a Given Language

STEP-BY-STEP METHOD:

Step 1: Identify the pattern

- What makes strings belong to the language?
- Are there equal counts of symbols?
- Is there nesting or recursion?

Step 2: Write base cases

- What are the simplest strings in the language?
- These become productions with only terminals

Step 3: Write recursive cases

- How do you build larger strings from smaller ones?
- These become productions with variables

Step 4: Test with examples

- Try deriving several strings
- Make sure all valid strings can be derived
- Check that invalid strings cannot be derived

EXAMPLE: Language $L = \{0^n 1^m 2^m \mid n \geq 0, m \geq 1\}$

Step 1 - Pattern: Any number of 0s, then equal 1s and 2s Step 2 - Base: 12 (smallest string with $m=1, n=0$) Step 3 - Recursive: Add 0s at front, or wrap 1-2 pairs Step 4 - Grammar:

$S \rightarrow AS \mid A \ A \rightarrow 1A2 \mid 12$

Testing:

- "12": $S \Rightarrow A \Rightarrow 12 \checkmark$
- "1122": $S \Rightarrow A \Rightarrow 1A2 \Rightarrow 1122 \checkmark$
- "0012": $S \Rightarrow 0S \Rightarrow 0A \Rightarrow 012 \checkmark$

15.2 Performing a Derivation

STEP-BY-STEP METHOD:

Step 1: Start with the start symbol

Step 2: Choose leftmost (or rightmost) variable to replace

Step 3: Select an appropriate production rule

Step 4: Apply the production

Step 5: Repeat until only terminals remain

Step 6: Write the complete derivation sequence

EXAMPLE: Derive "0101" from $A \rightarrow 0A0 \mid 1A1 \mid \epsilon$

Leftmost derivation:

Step 1: A (start) Step 2: $A \Rightarrow 0A0$ (apply production $A \rightarrow 0A0$) Step 3: $0A0 \Rightarrow 01A10$ (apply production $A \rightarrow 1A1$) Step 4: $01A10 \Rightarrow 0110$ (apply production $A \rightarrow \epsilon$)

Wait, this gives "0110" not "0101". Let me try again.

Actually, "0101" is NOT a palindrome, so it cannot be derived! This is correct.

For "010" (which IS a palindrome):

Step 1: A Step 2: $A \Rightarrow 0A0$ Step 3: $0A0 \Rightarrow 010$ (apply $A \rightarrow 1$)

Complete: $A \Rightarrow 0A0 \Rightarrow 010 \checkmark$

15.3 Drawing a Parse Tree

STEP-BY-STEP METHOD:

Step 1: Put start symbol at root

Step 2: For each production used in derivation:

- Create children for the variable node
- Label children with right-hand side symbols

Step 3: Continue until all leaves are terminals

Step 4: Verify yield matches target string

EXAMPLE: Parse tree for "01" from $S \rightarrow 01 \mid 0S1$

Step-by-step construction:

Step 1: Start with S at root

S

Step 2: Apply production $S \rightarrow 01$

S

/ \

0 1

Step 3: All leaves are terminals - DONE!

Yield: 0 1 = "01" ✓

EXAMPLE 2: Parse tree for "0011" from $S \rightarrow 01 \mid 0S1$

Step-by-step construction:

Step 1: Start with S at root

S

Step 2: Apply production $S \rightarrow 0S1$

S

/|\

0 S 1

Step 3: Apply production $S \rightarrow 01$ to the S child

S

/|\

0 S 1

/|\

0 1

Step 4: All leaves are terminals - DONE!

Yield: 0 0 1 1 = "0011" ✓

15.4 Checking for Ambiguity

STEP-BY-STEP METHOD:

Step 1: Pick a string from the language

Step 2: Try to find TWO different derivations

Step 3: If you find two different:

- Leftmost derivations, OR
- Rightmost derivations, OR
- Parse trees Then the grammar is AMBIGUOUS

Step 4: Show both derivations/trees explicitly

EXAMPLE: Is $S \rightarrow SS \mid (S) \mid ()$ ambiguous?

Test string: "()()"

Derivation 1: $S \Rightarrow SS \Rightarrow ()S \Rightarrow ()()$

Derivation 2: $S \Rightarrow SS \Rightarrow S() \Rightarrow ()()$

Both are leftmost derivations! They differ in which S we expand first.

Conclusion: Grammar is AMBIGUOUS